NO FA – Authentication Bypass via Leaked Credentials & Weak 2FA Implementation

1. Challenge Information

- Platform: PicoCTF
- Category: Web Exploitation
- Difficulty: Medium
- Link: https://play.picoctf.org/practice/challenge/765
- Vulnerability Type: Authentication Bypass
- Primary Issues: Weak Two-Factor Authentication (2FA) & Sensitive Data Exposure
- Related Standards:
    + OWASP A01: Broken Access Control
    + OWASP A02: Cryptographic Failures
    + CWE-287: Improper Authentication
    + CWE-522: Insufficiently Protected Credentials

2. Executive Summary

This challenge demonstrates a critical authentication bypass caused by a combination of **leaked credential data** and **insecure 2FA implementation**.
An attacker can extract hashed credentials from a provided database file and recover the admin password using publicly available hash databases. Although 2FA is implemented, it is fundamentally flawed: the One-Time Password (OTP) is stored client-side within a Flask session cookie. By decoding the session cookie, the attacker can retrieve the OTP without brute force, effectively bypassing the second authentication factor. This vulnerability highlights improper handling of authentication mechanisms and failure to securely manage sensitive data, leading to full account compromise.

3. Reconaissance & Enumeration

The challenge provides:
- A target web application
- Source code (`app.py`)
- Database file (`users.db`)

3.1. Source Code Analysis

Author. Alex Ngo

Reviewing `app.py` reveals:

```
@app.route("/")
def home():
    if 'username' not in session or session['logged'] == 'false':
        flash('Please login to access this page', 'red')
        return redirect(url_for('login'))

    flag = "No flag for you!!"
    if session.get('username') == 'admin':
        flag = os.getenv('FLAG')

    return render_template("index.html", flag=flag)

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        user = db.get_user_by_username(username)
        if user and hashlib.sha256(password.encode()).hexdigest() == user['password']:
```

```
            if user['two_fa']:
                # Generate OTP
                otp = str(random.randint(1000, 9999))
                session['otp_secret'] = otp
                session['otp_timestamp'] = time.time()
                session['username'] = username
                session['logged'] = 'false'
                # send OTP to mail ---
                return redirect(url_for('two_fa'))
            else:
                session['username'] = username
                session['logged'] = 'true'
                flash('Login successful!', 'green')
                return redirect(url_for('home'))
        else:
            flash('Invalid username or password', 'red')
    return render_template('login.html')

@app.route('/two_fa', methods=['GET', 'POST'])
def two_fa():
    if request.method == 'POST':
        otp = request.form['otp']
        stored_otp = session['otp_secret']
        timestamp = session.get('otp_timestamp')
        if stored_otp and otp == stored_otp and (time.time() - timestamp) < 120:
            session['logged'] = 'true'
            flash('Login successful!', 'green')
            return redirect(url_for('home'))
        else:
            flash('Invalid OTP or OTP expired', 'red')
            return render_template('2fa.html')
    else:
        return render_template('2fa.html')
```

- Authentication logic for user login
- Admin account grants access to the flag
- Passwords are stored using SHA-256 hashing

Author: Alex Ngo

- A 2FA mechanism is implemented for certain users

If a user has `two_fa = TRUE`, they are redirected to: /two_fa

3.2. 2FA Mechanism Analysis

The OTP generation logic contains several weaknesses:
- OTP range: **1000–9999 (only 9000 possible values)**
- OTP is stored inside a **Flask session cookie**
- Session is sent back to the client

This indicates **client-side exposure of sensitive authentication data**

3.3. Database Analysis

The `users.db` file was analyzed using: 'strings users.db'

```
SQLite format 3
Ytablesqlite_sequencesqlite_sequence
CREATE TABLE sqlite_sequence(name,seq)
tableusersusers
CREATE TABLE users (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                username TEXT UNIQUE NOT NULL,
                email TEXT NOT NULL,
                password TEXT NOT NULL,
                two_fa BOOLEAN NOT NULL DEFAULT 0
            ))
indexsqlite_autoindex_users_1users
joseph.colejoseph.cole@nfa.com49a57175de704a0ec2a006746d20d375814581bb35552ce0a0b13683426fd232k
```

```
        adminiamadmin@nfs.comc20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabda079ca9e5ab67e
emily.brownemily.brown@nfa.com9e85668a071a595fe9222725bfb591cdaa0d880e3a7c7de1d9ddd3d4b7d08772g
```

This revealed:

- Usernames
- SHA-256 hashed passwords

The **admin account hash** was successfully identified.

4. Exploitation & Attack Vector

4.1. Credential Recovery

The extracted SHA-256 hash was submitted to an online hash database: https://hashes.com

The plaintext password for the admin account was successfully recovered.

✔ Found:

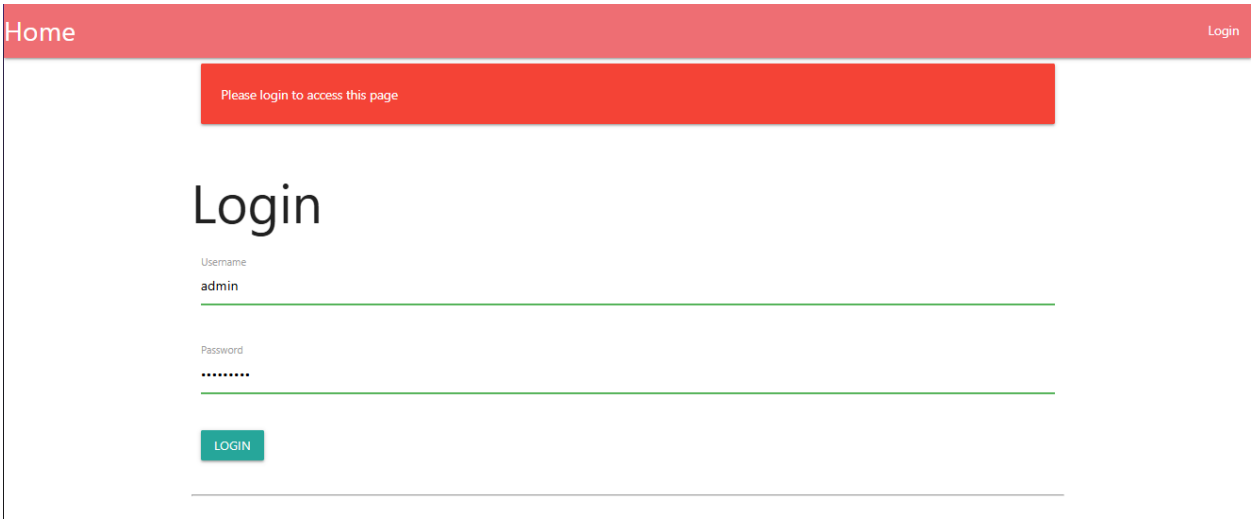
```
c20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabda079ca9e5ab67:apple@123
c20fa16907343eef642d10f0bdb81bf629e6aaf6c906f26eabda079ca9e5ab67:apple@123
```

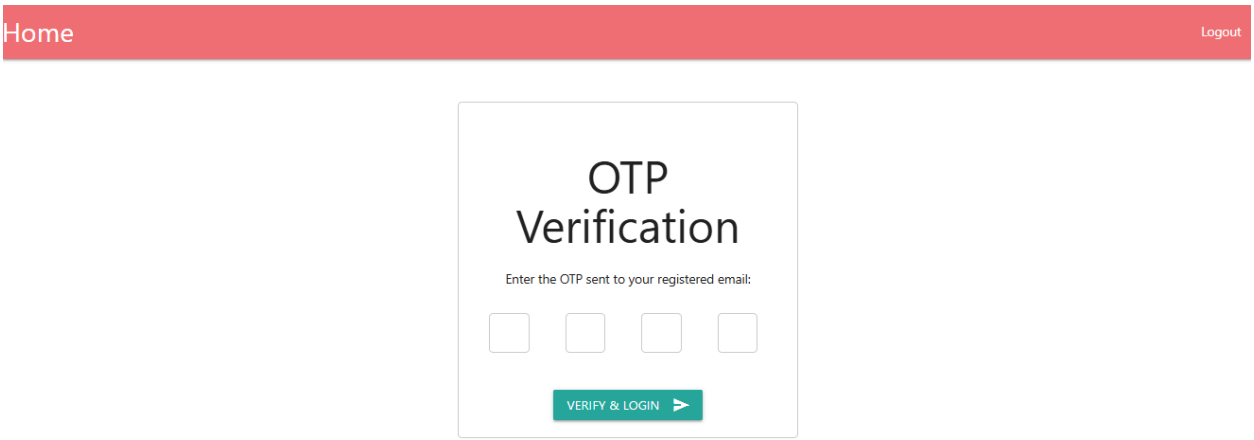
4.2. Initial Authentication

Using recovered credentials:

Username: admin
Password: apple@123

Login was successful, but redirected to the **2FA verification page**.





4.3. Session Cookie Analysis

Using browser Developer Tools (Network tab), the session cookie was inspected.

Cookie	session=.eJwty0sKgCAQANC7zFoiUzO9TAw1ieAPtVV091y0ffAeCNk5OsHChaERMMi97I2OSn2gnDf-W_eRWsdYwHKtlTGLFnriKxdSKgZ3o5ow0kh4Rp_g_QBF2Rxh.adt_oQ.Vc7ulcjxp7JzkW7fT69QX1sV6Xl
Host	foggy-cliff.picoctf.net:50702
Referer	http://foggy-cliff.picoctf.net:50702/login

Key observations:

- Cookie starts with "." → Flask session format
- Structure consists of 3 parts:
    1. Encoded payload (contains OTP)
    2. Timestamp
    3. Digital signature

The payload is:

- Base64 encoded
- Compressed using zlib

4.4. OTP Extraction

A public tool was used to decode the Flask session:

- https://github.com/noraj/flask-session-cookie-manager

Command example:

python3 flask_session_cookie_manager.py decode -c <SESSION_COOKIE>

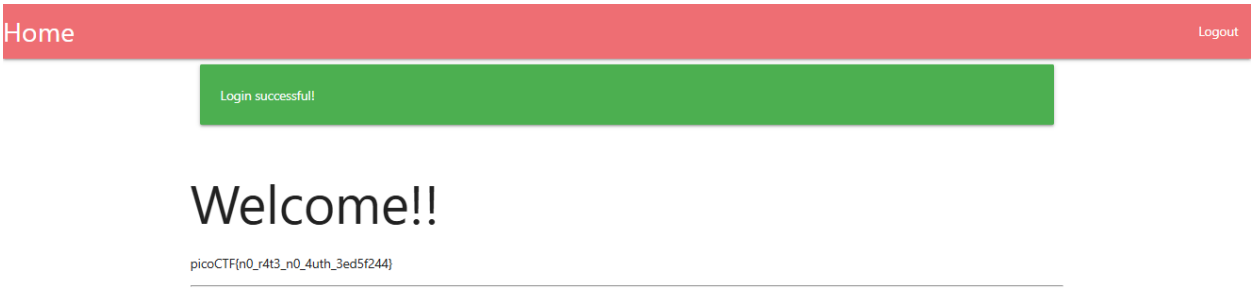
Decoded payload revealed:

```
b'{"logged":"false","otp_secret":"4081","otp_timestamp":1775992737.1613445,"username":"admin"}'
```

4.5. 2FA Bypass

The extracted OTP was entered into the 2FA page.

Authentication successful & Flag retrieved



5. Root Cause Analysis

The vulnerability exists due to multiple critical design flaws:

5.1 Sensitive Data Exposure

- Credentials stored in accessible database file
- Weak protection of password hashes

5.2 Broken 2FA Implementation

- OTP stored client-side in session cookie
- No server-side validation integrity

5.3 Insecure Session Design

- Predictable structure
- Decodable without secret key protection

5.4 Weak OTP Design

- Low entropy (4-digit numeric)

6. Remediation & Best Practices

To prevent such vulnerabilities:

6.1. Secure Credential Storage

- Use strong hashing algorithms (e.g., bcrypt, Argon2)
- Never expose database files publicly

6.2. Proper 2FA Implementation

- Store OTP **server-side only**
- Use time-based OTP (TOTP)
- Expire OTPs after short duration

6.3. Secure Session Management

- Do not store sensitive data in client-side cookies
- Use server-side session storage (Redis, DB)
- Encrypt and sign session data securely

6.4. Improve OTP Strength

- Increase entropy (6–8 digits or alphanumeric)

- Implement rate limiting and brute-force protection

6.5. Conduct Security Testing

- Perform code reviews before deployment
- Use automated security scanning tools